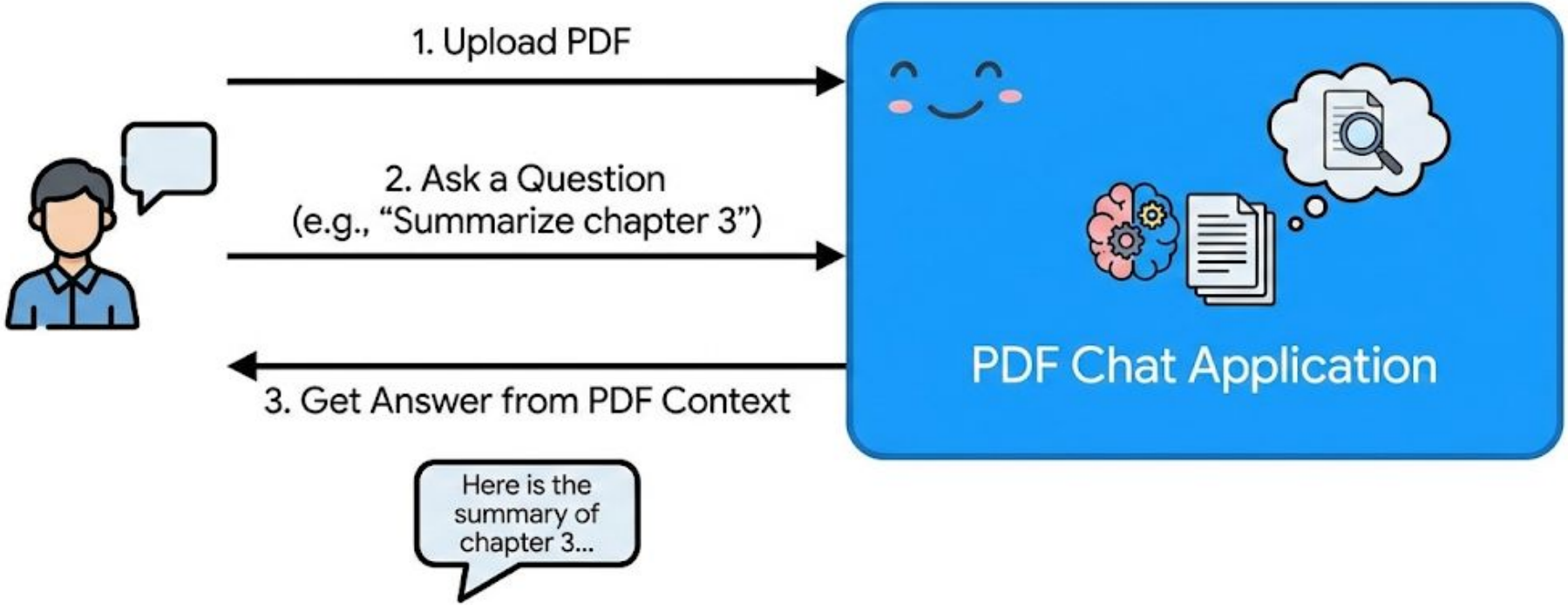
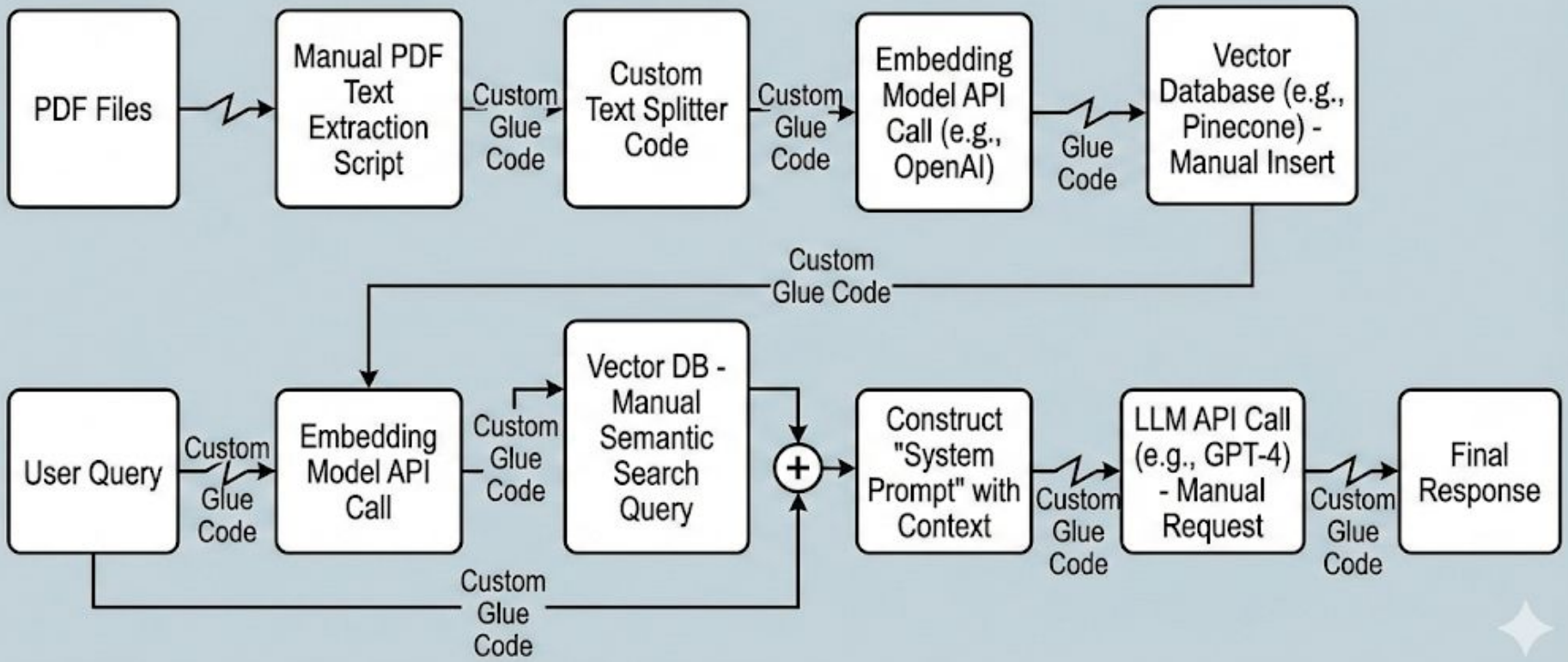
Langchain

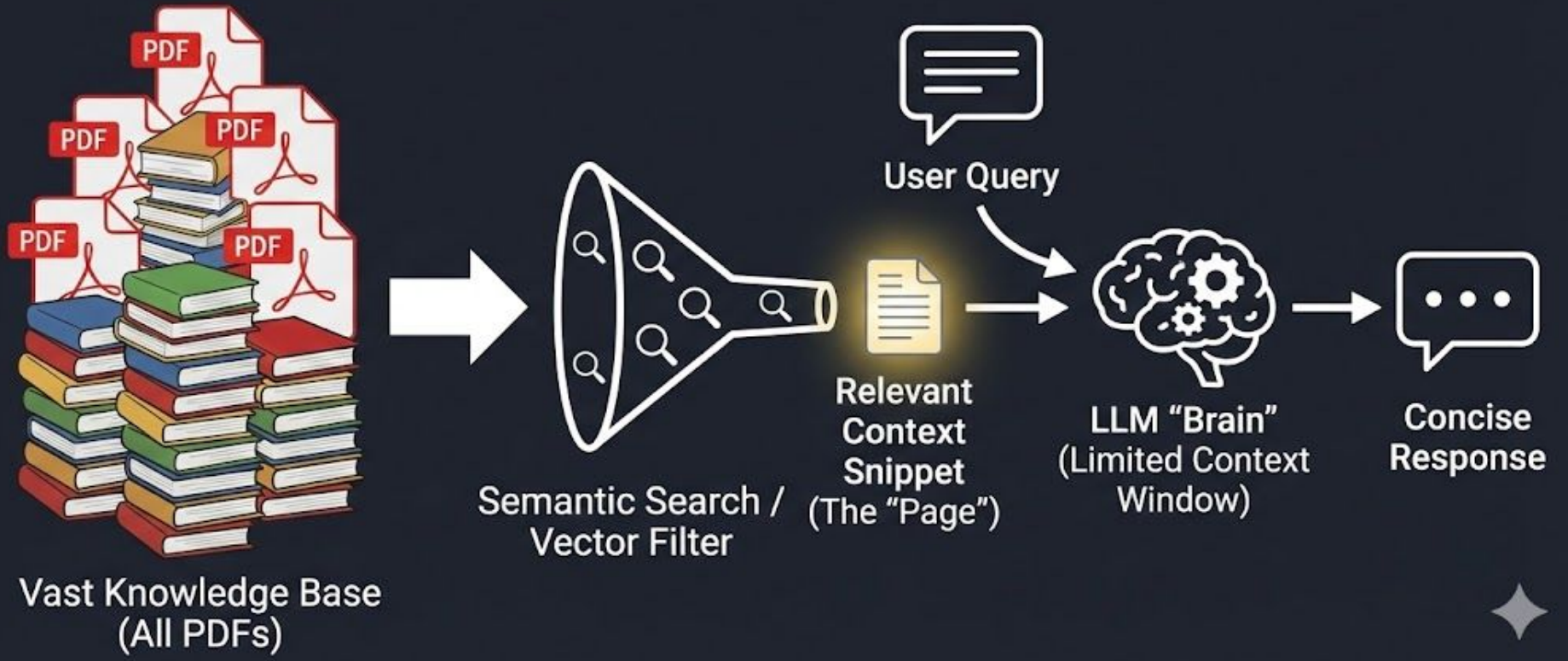
The Goal: A “Chat with Your PDF” Application



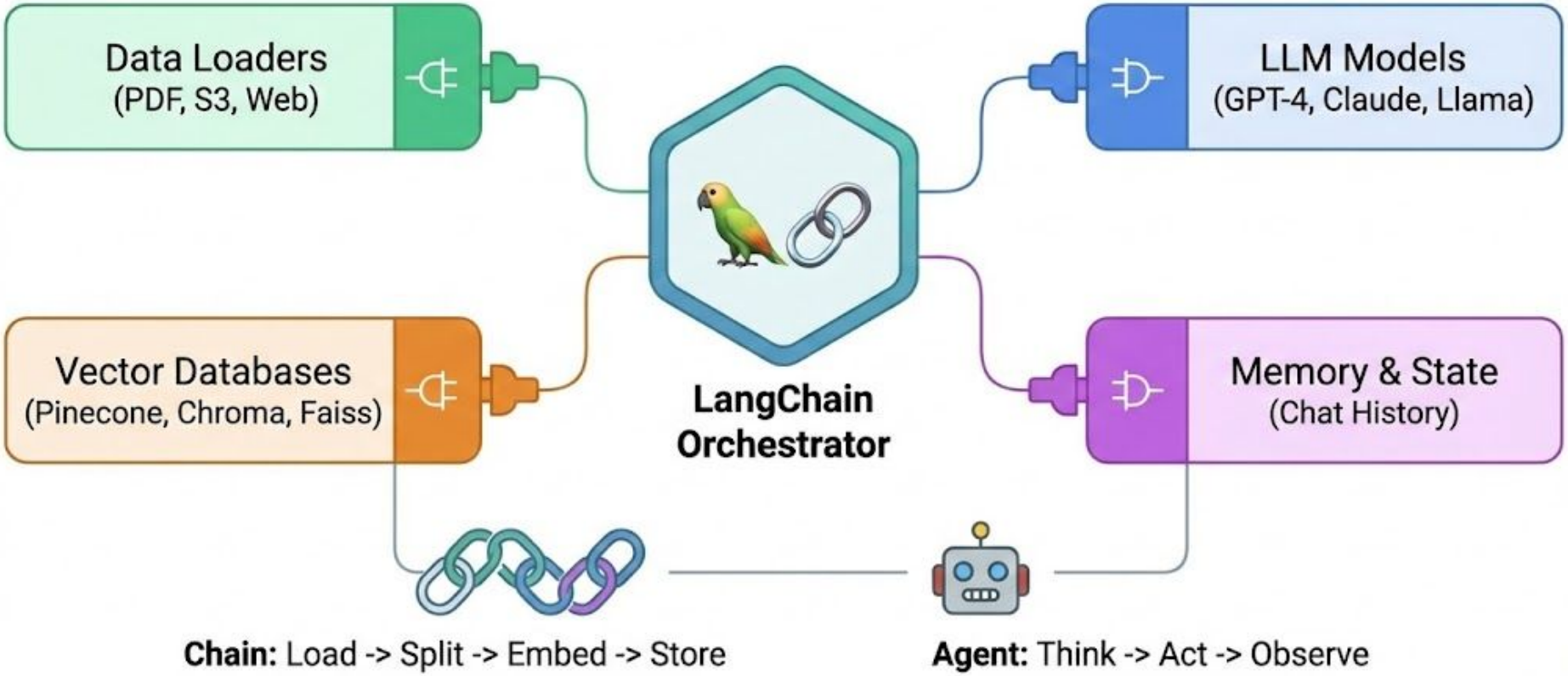
The “Manual” Workflow (The Problem)



The “Brain” Gap: Why Semantic Search?

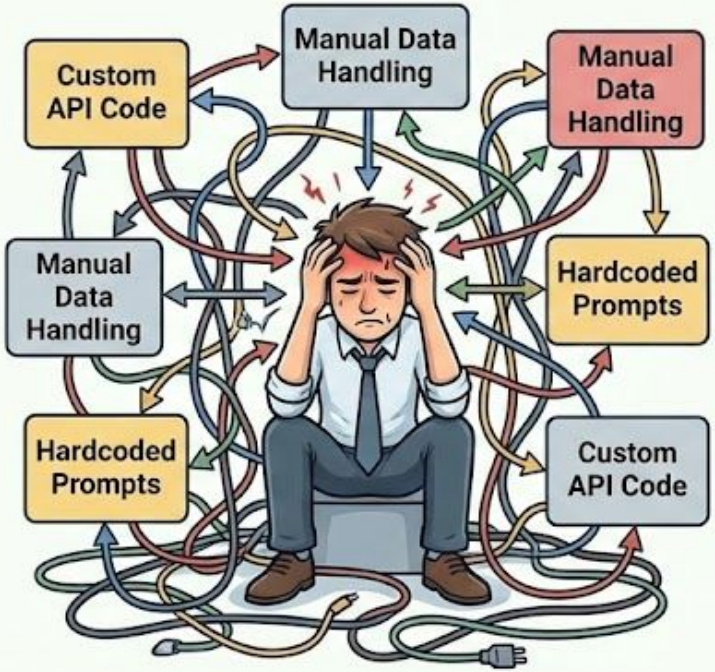


The LangChain Solution: The "Plugin" Architecture



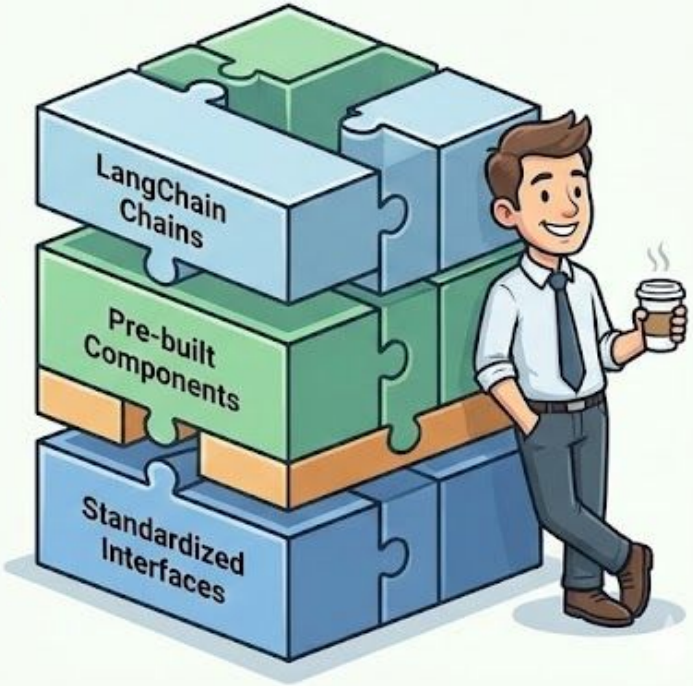
Before vs. After: The LangChain Effect

BEFORE: Manual Development
(The Hard Way)



Standardization & Efficiency

AFTER: LangChain Development
(The Fast Way)



What is Langchain

- LangChain is a framework to build real-world applications using LLMs by chaining together prompts, memory, tools, and external APIs.
- If LLMs like GPT are the brains, LangChain is the nervous system connecting them to tools, memory, and logic.
- “LangChain helps developers build modular, composable, and agentic LLM applications — not just chatbots.”

Need of langchain

- LLMs alone can:

  Predict text

  Answer questions

  Complete prompts

- But they can’t:

  Use tools (e.g., calculator, search engine)

  Access memory over conversations

  Make decisions (e.g., “Should I use a tool or just answer?”)

  Combine multiple prompts or steps together

  LangChain enables this!

Langchain Components

Component	What it does	Example
LLM	Calls models like GPT, Claude, etc.	`ChatOpenAI()`
Prompt	Prompt templates with variables	`PromptTemplate()`
Chain	Sequential execution of steps	`LLMChain()`
Tool	Interface to calculators, web search	`Tool()`
Agent	Decides which tool or chain to use	`initialize_agent()`
Memory	Keeps track of conversation	`ConversationBufferMemory()`

LLMs that work with Langchain

Langchain supports many models

- OpenAI GPT
- Anthropic Claude
- Hugging Face Transformers
- Cohere
- Google PaLM, Gemini (via APIs)
- Ollama (for local models)

Parsers in Langchain

- In LangChain, when you call an LLM directly, you usually get something like:
- **AIMessage(content="Here is the answer...")**
- This is not a string. It’s a message object.
- Now imagine:
  - You want to pass this output to another chain
    - Or store it in a DB
    - Or parse it as JSON later
    - Or run evals on it
    - Or just print() without worrying
- That’s where parsers enter.

Without StrOutputParser

```
chain = prompt | llm
result = chain.invoke('hello')
```

```
AIMessage(
  ...
  content="Hi there!",
  response_metadata: {
    "token_usage": {"prompt_tokens": 10, "completi
    "model_name": "aimev chain",
    "finish_reason": "stop"
  },
  id: "run-12345-abcdef",
  ...
)
```

Result type:
AIMessage(content='Hi there!')

So every time:
result.content

→

```
result.content
```

With StrOutputParser

```
from langchain_core.output_parsers import StrOutputParser

chain = prompt | llm | StrOutputParser()
result = chain.invoke('hello')
```

Hi there!

Result: 'Hi there!'

Clean string. Nothing extra.

Parsers in Langchain

- **Predictable Output Type:**

```
chain1 = prompt1 | llm | StrOutputParser()
chain2 = chain1 | prompt2 | llm
```

  - If you skip parser:
  - chain2 might receive AIMessage
  - prompt2 expects str
  - Type mismatch / silent bugs

Parsers in Langchain

- **Future-Proofing:**
- Today you want string.
- Tomorrow you want:
  - JSONOutputParser
  - PydanticOutputParser
  - Custom regex parser
- If you already use a parser: swapping becomes trivial.

Parsers in Langchain

- **Clean Separation of Responsibilities:**
  - LangChain philosophy:
  - LLM → generate
  - Parser → interpret
  - Chain → orchestrate
- Mixing these makes debugging painful.

Langchain Chains

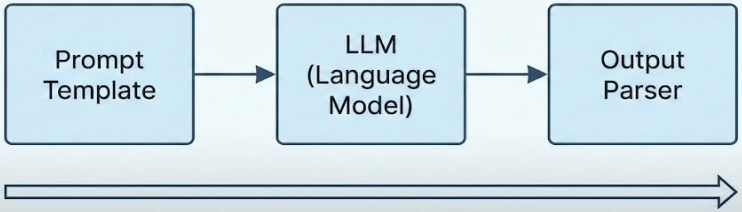
- A Chain in LangChain is essentially a sequence of calls to components (like LLMs, tools, or data processing steps) that allows you to automate complex tasks by linking them together.
- In the old days (a few months ago), you’d just send a prompt to an LLM and get a result. But real-world applications require multiple steps.
- If you’re building a house, you don't just "make a house." You lay the foundation, build the walls, then roof it. Each step depends on the previous one. A Chain is that construction plan.

Langchain Chains

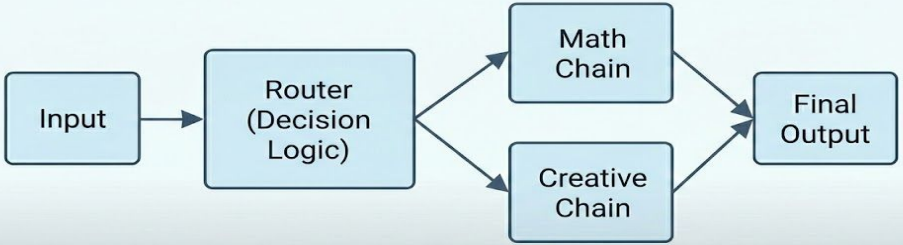
- While there are "Legacy" chains, the industry has moved to **LCEL (LangChain Expression Language)**. It uses the pipe operator (|) to flow data from one component to the next.
- **Basic Chain:**

  A Basic Chain is a single, atomic operation. It takes a Prompt Template, passes it to an LLM, and then uses an Output Parser to format the response. This is the "Hello, World!" of LangChain.

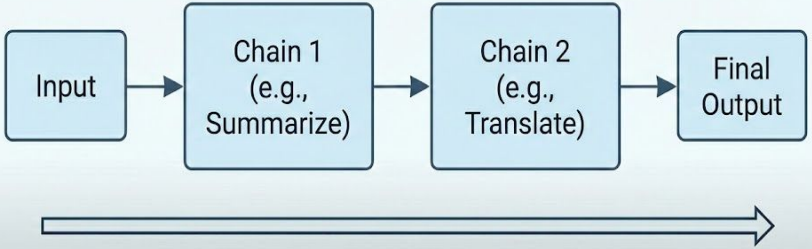
Basic Chain: The Fundamental Unit



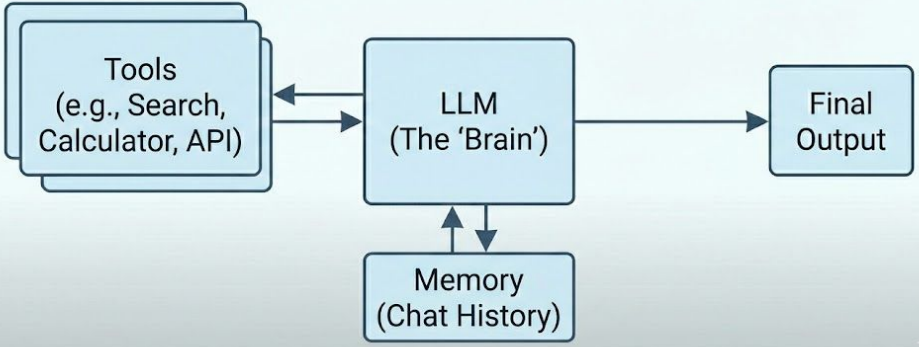
Router Chain: Branching Logic



Sequential Chain: Linking Steps Together



Agent: Dynamic Execution with Tools



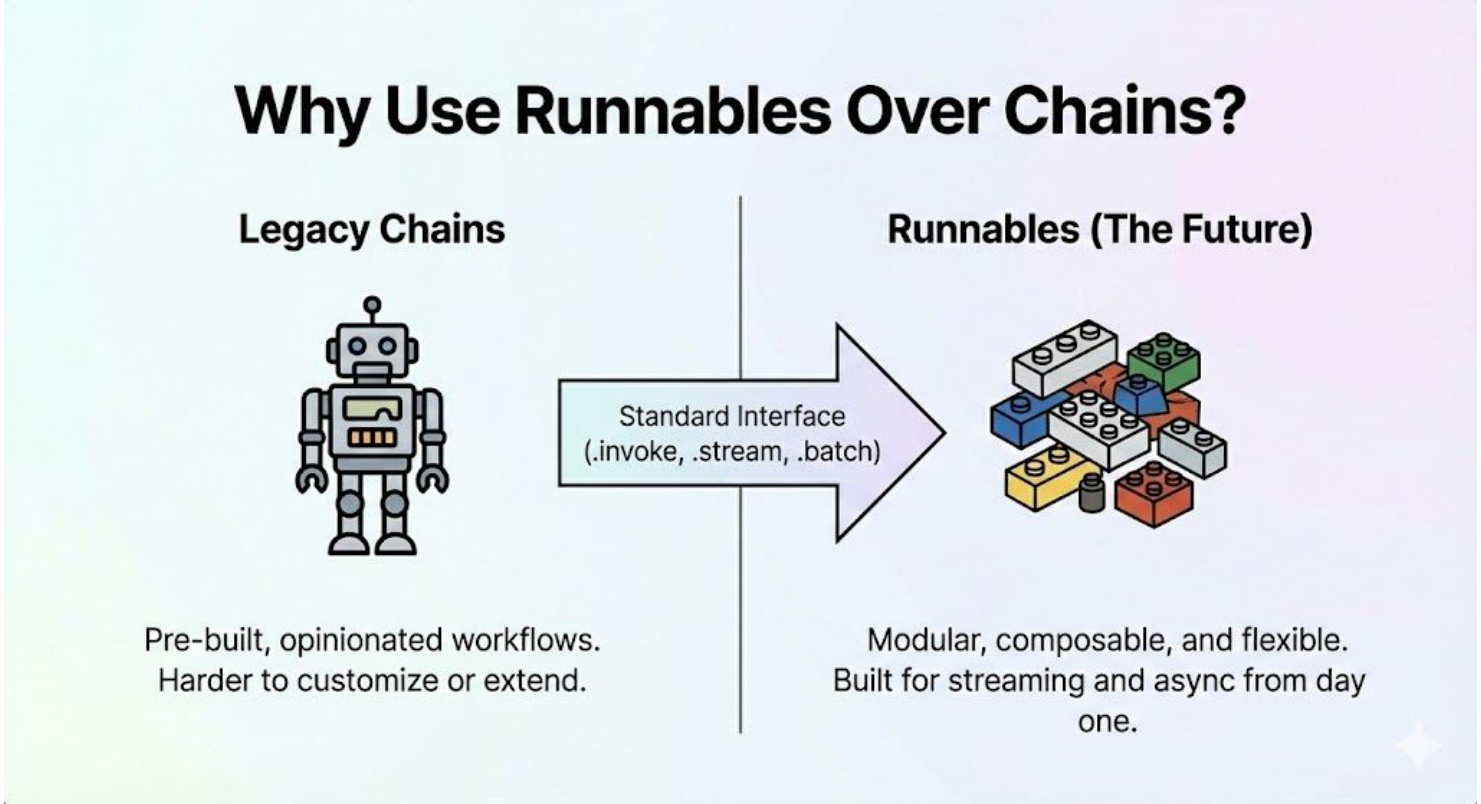
Langchain Runnables

Runnables are the core abstraction in modern LangChain (v0.1+) — anything that can be "run" with standard input/output interfaces. They replace the rigid Chain class with a composable, pipe-able architecture.

They provide a unified interface for all components.

Langchain chains are in-fact work using runnables themselves.

Langchain Runnables



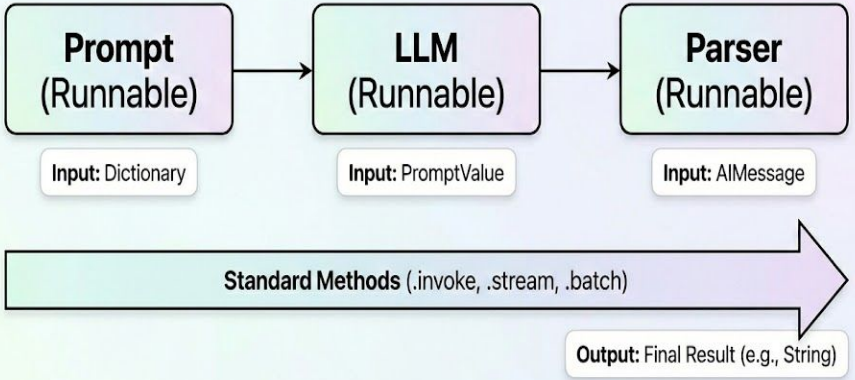
Langchain Runnables

Common Types of Runnables

Prompts
PromptTemplate, ChatPromptTemplate

Models
ChatModel, LLM

Parsers
StrOutputParser, JsonOutputParser

Flow Control
RunnablePassthrough, RunnableParallel

What Runnables Do: The Standard Interface